AI Meets Control Theory

From Classical Control to Intelligent Autonomous Systems

Living Technical Report — M12 — Real Multi-Body Systems & Real-Time Iteration NMPC

Generated by the project team
Lead: puma | **Code:** puma, toku | **Docs/Research:** lava, famo

September 4, 2026

Abstract

This report documents the culmination of the **AI Meets Control Theory** project. Across 10 curriculum modules, 24 rigorous from-scratch benchmark experiments, and 373 verified unit tests, we evaluate and compare classical control, modern state-space methods, Kalman filtering, optimal control, constrained Model Predictive Control (MPC), real-time iteration nonlinear MPC (iLQR / RTI-NMPC), adaptive control, Euler–Lagrange robot manipulators, non-holonomic mobile robotics, deep reinforcement learning, and hybrid AI + control safety shields on physical dynamical systems, under the core principle: *build every method from scratch, simulate it, visualise it, and compare it honestly.*

Contents

1 Introduction and Scope

The central question of the project is: **when should we use classical control, when should we use AI, and when should we use both?** Rather than assuming a neural network or an RL agent is the answer, every problem is first attacked with PID, state feedback, observers, LQR, Kalman filtering (linear, EKF, UKF), system identification, adaptive control (MRAC, Slotine–Li), nonlinear energy shaping, computed torque, constrained linear MPC, real-time iteration nonlinear MPC (iLQR / RTI-NMPC), sampling-based nonlinear MPC with obstacle avoidance, learned neural dynamics, deep reinforcement learning (DQN, PPO), behavioral cloning (BC), and safety shields, and each method is measured against the others under identical initial conditions, sensor suites, noise profiles, disturbances, and actuator constraints.

Every topic follows the standardized engineering cycle:

THEORY $\longrightarrow$ DERIVATION $\longrightarrow$ IMPLEMENTATION $\longrightarrow$ SIMULATION $\longrightarrow$ VISUALISATION $\longrightarrow$ VALIDATION $\longrightarrow$ COMPARISON $\longrightarrow$ EXPERIMENT

Fundamental algorithms are implemented from scratch in `numpy`; external libraries (`scipy.linalg`, `scipy.signal`, `control`, `gymnasium`, `stable-baselines3`) are used strictly for verification and benchmarking compliance.

2 Framework Architecture

The reusable library is the Python package `aimct` (`src/aimct/`). Its architecture separates distinct concerns behind minimal, strongly-typed interfaces so that dynamical systems, controllers, state estimators, neural network models, reinforcement learning environments, safety shields, and the simulator compose seamlessly.

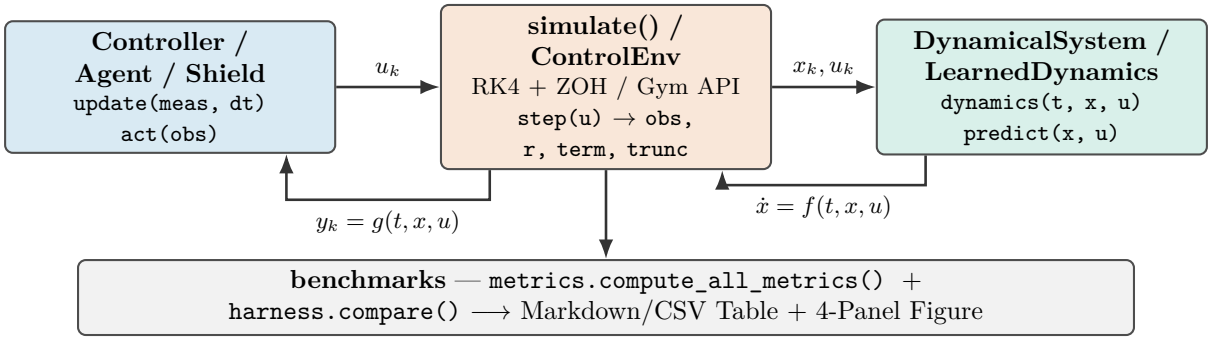

Figure 1: Core framework architecture. The numerical simulation engine drives a controller (`update()`) and continuous plant or learned neural surrogate (`dynamics()`) via fixed-step 4th-order Runge–Kutta with Zero-Order Hold (ZOH).

2.1 Package Layout

```
src/aimct/
  systems/      DynamicalSystem base + LinearSystem, MassSpringDamper,
                Pendulum, CartPole, PlanarQuadrotor (Crazyflie 2.0), DCMotor,
                DifferentialDriveRobot (unicycle + lag), TwoLinkArm (Euler-Lagrange)
  simulate.py   rk4_step(), simulate() -> Trajectory(t, x, u, y)
  controllers/  PID, StateFeedback, LQR, ObserverFeedback, MRAC, ComputedTorque,
                EnergyShapingSwingUp, HybridSwingUpLQR,
                LinearMPC (with preview), SamplingMPC (CEM + obstacles),
                ILQR (trajectory optimiser + real-time-iteration NMPC)
  estimation/   LuenbergerObserver, KalmanFilter (LQE/FARE), DiscreteKalmanFilter,
                ExtendedKalmanFilter, UnscentedKalmanFilter, observability_matrix
  trajectories/ Lemniscate, Spline, Minimum-Jerk Polynomials, Dubins paths
  sysid/        least_squares_id, dmdc, to_continuous (block logm), prediction_error
  ml/           MLP (backprop + Adam), LearnedDynamics (grey-box / residual)
  rl/           ControlEnv, Discretizer, QLearning, DQN, REINFORCE, PPO, BC
  hybrid/       ShieldedController (switch/filter blends, predicate helpers)
  viz/          SystemArtist contract (Pendulum, CartPole, TwoLinkArm, DiffDrive,
                PlanarQuadrotor), animate() replay generator, Sandbox live GUI
  benchmarks/   metrics.py (13 metrics), harness.py, sweep.py, challenge.py (ICC
    engine),
                tracking.py (path following benchmark), capstone_scoring.py
  plot_style.py Okabe-Ito palette + publication-ready 4-panel comparison figure
```

3 Current Capabilities

Table 1 summarises the implemented components, their mathematical scope, and the corresponding verification test coverage as of M12 — Real Multi-Body Systems & Real-Time Iteration NMPC.

Component	What it provides	Tests
`aimct.systems`	Continuous-time ODE models with a common `dynamics(t, x, u)` interface, full-state or output projections, and analytical/numerical `linearize()`. Ships: `LinearSystem`, `MassSpringDamper`, `Pendulum`, `CartPole`, `PlanarQuadrotor` (Crazyflie 2.0), `DCMotor`, `DifferentialDriveRobot` (unicycle + motor lag), `TwoLinkArm` (Euler–Lagrange with wrist payload).	✓
`aimct.simulate`	Fixed-step RK4 integrator with zero-order hold; `Trajectory` container; hard actuator saturation; measurement and disturbance injection hooks.	✓
`aimct.controllers.PID`	From scratch: filtered derivative-on-measurement ($D(s) = \frac{K_d s}{\tau_d s+1}$), conditional-integration anti-windup clamping, setpoint weighting.	✓
`aimct.controllers.StateFeedback`	Arbitrary closed-loop pole placement via Ackermann's formula (SISO) with feedforward steady-state gain scaling k_r.	✓
`aimct.controllers.LQR`	Infinite-horizon optimal regulator; Continuous Algebraic Riccati Equation (CARE) solved from scratch via Hamiltonian Schur decomposition, cross-checked with `scipy.linalg`.	✓
`aimct.controllers.MRAC`	Model Reference Adaptive Control: Lyapunov-derived parameter adaptation tracking a reference model under drifting or uncertain plant parameters.	✓
`aimct.controllers.ILQR`	Iterative Linear Quadratic Regulator (iLQR) trajectory optimiser with quadratic-cost expansion, backward Riccati pass, forward line search, and Real-Time-Iteration (RTI-NMPC) wrapper.	✓
`aimct.controllers.ObserverFeedback`	Unified output feedback combining any state estimator (Luenberger, linear KF, EKF, UKF) with static feedback gain $u = -K\hat{x}$.	✓
`aimct.controllers.SwingUp`	Nonlinear Spong energy shaping (`EnergyShapingSwingUp`) with partial feedback linearisation, plus finite-state hysteresis supervisor (`HybridSwingUpLQR`).	✓
`aimct.controllers.LinearMPC`	Constrained linear MPC via condensed receding-horizon QP, active-set solver (`_qp.solve_qp`), DARE terminal cost, soft state constraints, and trajectory preview.	✓
`aimct.controllers.SamplingMPC`	Sampling-based nonlinear MPC using the Cross-Entropy Method (CEM), vectorized batch trajectory rollouts, variance floor, CARE terminal weight, and online non-convex obstacle avoidance.	✓
`aimct.estimation`	State estimation suite: `observability_matrix`, `is_observable`, `place_observer`, `LuenbergerObserver`, `solve_fare`, `KalmanFilter`, `DiscreteKalmanFilter`, `ExtendedKalmanFilter`, and `UnscentedKalmanFilter` ($2n + 1$ scaled sigma points, Merwe parameters, zero Jacobian linearisation).	✓
`aimct.trajectories`	Trajectory generators: Lemniscate, minimum-jerk polynomials, Dubins paths, parametric splines.	✓
`aimct.sysid`	Data-driven system identification: `least_squares_id`, `dmdc` (rank-truncated SVD), `to_continuous` (exact block matrix logarithm), `model_mismatch`.	✓
`aimct.ml`	Machine learning suite: multi-layer perceptron (`MLP`) with hand-written backpropagation + Adam, and residual / grey-box forward dynamics predictor (`LearnedDynamics` with `base_step`).	✓
`aimct.rl`	Reinforcement learning suite: Gymnasium `ControlEnv`, `Discretizer`, Tabular `QLearning`, `DQN` (experience replay + target net), `REINFORCE`, `PPO` (clipped surrogate objective + GAE), and Behavioral Cloning (`BC`).	✓
`aimct.hybrid`	Hybrid AI + control safety shields: `ShieldedController` (switch and filter action blends, predicate helpers `box_predicate`, `barrier_predicate`, and audit logging).	✓

4 Theory: Modelling and Simulation

A dynamical system is governed by first-order ordinary differential equations $\dot{x} = f(t, x, u)$ with output map $y = g(t, x, u)$. Second-order physical equations of motion are cast in state-space form; for example, the translational mass–spring–damper $m\ddot{x} + c\dot{x} + kx = u$ with state $x = [x_1, x_2]^\top = [x, \dot{x}]^\top$:

$$\frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u.$$

Jacobian Linearisation. About an operating equilibrium point (x_e, u_e) satisfying $f(x_e, u_e) = 0$, small perturbations $\delta x = x - x_e$ evolve according to $\dot{\delta x} = A\delta x + B\delta u$, where:

$$A \triangleq \left.\frac{\partial f}{\partial x}\right|_{(x_e, u_e)}, \qquad B \triangleq \left.\frac{\partial f}{\partial u}\right|_{(x_e, u_e)}.$$

The `DynamicalSystem` base class evaluates A, B numerically via central differencing ($\epsilon = 10^{-6}$), while concrete models implement analytical Jacobians.

Numerical Integration. The continuous dynamics are integrated over time steps $h = \Delta t$ using explicit 4th-order Runge–Kutta (RK4):

$$k_1 = f(t_k, x_k, u_k), \quad k_2 = f\left(t_k + \tfrac{h}{2}, x_k + \tfrac{h}{2}k_1, u_k\right), \quad k_3 = f\left(t_k + \tfrac{h}{2}, x_k + \tfrac{h}{2}k_2, u_k\right), \quad k_4 = f(t_k+h, x_k+hk_3, u_k$$

$$x_{k+1} = x_k + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

Holding control input $u(t) \equiv u_k$ constant across each interval $[t_k, t_k + h)$ reflects the physical Zero-Order Hold (ZOH) of digital microcontrollers.

5 Theory: Classical Control — PID Architecture

The continuous-time PID control law with filtered derivative is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t).$$

In discrete time with sampling interval h, derivative action is applied directly to the measured output with low-pass filter time constant $\tau_d = \frac{K_d}{NK_p}$ ($N = 10$) to prevent high-frequency noise amplification:

$$e_k = r_k - y_k, \qquad D_k = \frac{\tau_d}{\tau_d + h} D_{k-1} + \frac{K_d}{\tau_d + h}(y_k - y_{k-1}), \qquad u_{\text{unsat},k} = K_p e_k + I_k - D_k.$$

Anti-Windup Protection. When actuators reach physical limits ($|u| \le u_{\max}$), *conditional integration (clamping)* freezes the integrator update ($I_{k+1} = I_k$) whenever u_{unsat} is saturated and e_k has the same sign as u_k, preventing destructive overshoot upon setpoint arrival.

5.1 Worked Example: Stabilising an Unstable Plant (Experiment 03)

We evaluate the controller on an open-loop unstable plant with a Right-Half Plane (RHP) pole:

$$\ddot{y}(t) - \omega_0^2 y(t) = u(t) + d(t), \qquad \omega_0 = 2.0\,\text{rad/s}, \quad G(s) = \frac{1}{s^2 - 4}.$$

The system is tested under a unit step reference $r = 1.0\,\text{rad}$, a step input disturbance torque $d = 0.5\,\text{N}\cdot\text{m}$ applied at $t = 5.0\,\text{s}$, and actuator saturation $|u(t)| \le 10.0\,\text{N}\cdot\text{m}$.

Controller	Rise t_r [s]	Settle t_s [s]	Overshoot M_p	Steady e_{ss}	IAE	ITAE	Energy E_u	Sat. %
P-only ($K_p = 20$)	0.275	10.0	9.54×10^{9}%	6.04×10^{7}	4.77×10^{7}	4.53×10^{8}	984.3	97.62%
PD ($K_d = 5.66$)	0.389	10.0	28.19%	0.2812	2.811	13.63	316.3	1.60%
PID (no AW)	0.334	6.162	53.08%	3.90×10^{-5}	0.9274	1.043	262.2	5.55%
PID + Anti-Windup	0.362	6.161	**39.43%**	$\mathbf{3.90 \times 10^{-5}}$	**0.7979**	**0.876**	**245.1**	**1.61%**

Table 2: Benchmark results for Experiment 03 (regenerated from `experiments/03_pid_stabilizes_unstable/table.md`).

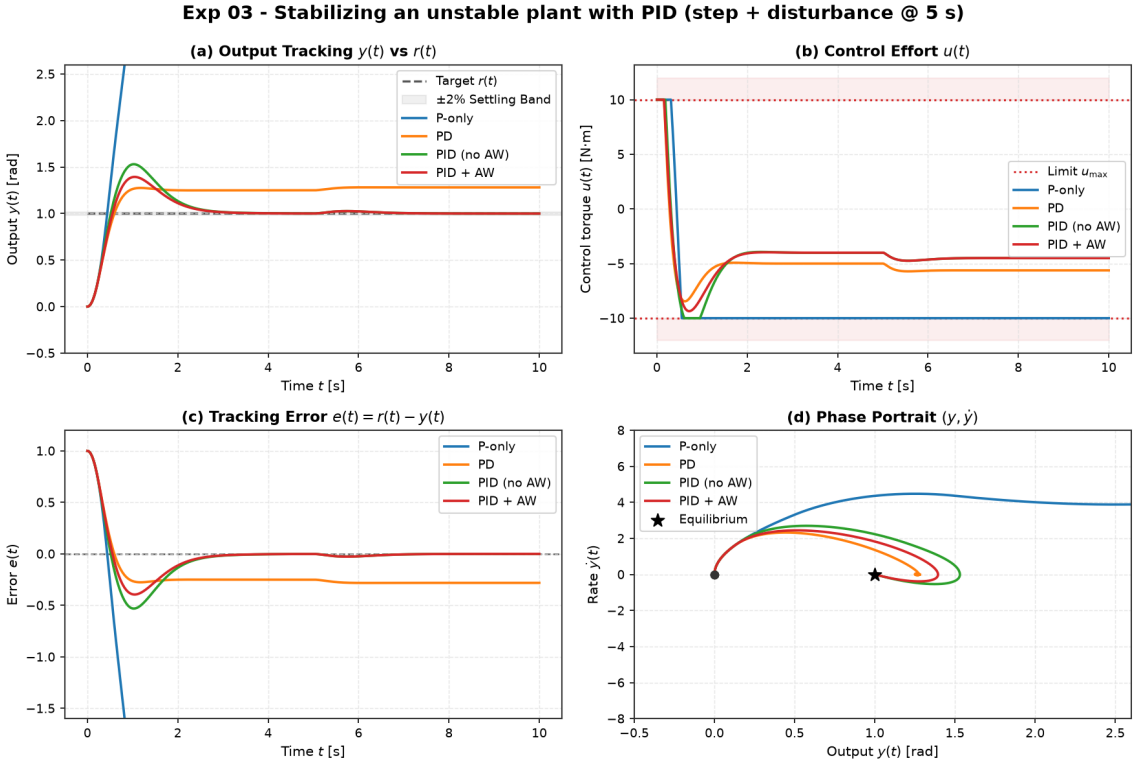

Figure 2: Experiment 03 benchmark comparison. (a) Output tracking $y(t)$, (b) Control effort $u(t)$ with saturation limits $\pm 10\,\mathrm{N \cdot m}$, (c) Tracking error $e(t)$, and (d) Phase portrait $(y, \dot{y})$.

6 Theory: Modern Control, Estimation, and Optimal Design

6.1 Pole Placement and LQR

For controllable linear systems $\dot{x} = Ax + Bu$, full-state feedback $u = -Kx + k_r r$ assigns arbitrary closed-loop poles $\sigma(A - BK)$. `StateFeedback` computes K via Ackermann's formula:

$$K = \begin{bmatrix} 0 & \dots & 0 & 1 \end{bmatrix} \mathcal{C}(A, B)^{-1} \Delta_{\text{des}}(A), \qquad k_r = -\frac{1}{C(A - BK)^{-1} B}.$$

Linear Quadratic Regulator (LQR). Rather than guessing pole locations, LQR computes the optimal feedback gain $K = R^{-1} B^\top P$ minimising the infinite-horizon quadratic cost:

$$J = \int_0^\infty \left(x(t)^\top Q x(t) + u(t)^\top R u(t) \right) dt, \qquad Q \succeq 0, \quad R \succ 0.$$

The kernel $P = P^\top \succ 0$ uniquely solves the **Continuous Algebraic Riccati Equation (CARE)**:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

We solve CARE from scratch via the stable invariant subspace of the Hamiltonian matrix:

$$\mathcal{H}_{\text{ham}} = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix} \in \mathbb{R}^{2n\times 2n}.$$

Continuous full-state LQR provides guaranteed robustness margins: gain margin $G_m \in [1/2, \infty)$ ($-6\,\text{dB}$ to $+\infty\,\text{dB}$) and phase margin $\Phi_m \geq 60^\circ$ in all channels.

6.2 Worked Example: Cart-Pole Regulation (Experiment 04)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^{2}s]	Peak $\lvert u\rvert_{\max}$ [N]	Peak $\lvert x\rvert_{\max}$ [m]	Status
Pole Placement	1.05	0.0171	2.34	8.71	**0.162**	Stable
LQR (Balanced)	1.05	0.0171	2.34	8.71	**0.162**	Stable
LQR (Soft)	1.70	0.0223	**0.959**	**3.08**	0.321	Stable
PID (Angle Only)	**0.20**	**0.0110**	8.76	18.00	**0.823**	Stable (Cart Drifts)

Table 3: Benchmark results for Experiment 04 on nonlinear Cart-Pole (from `experiments/04_lqr_vs_pole_placement_cartpole/table.md`).

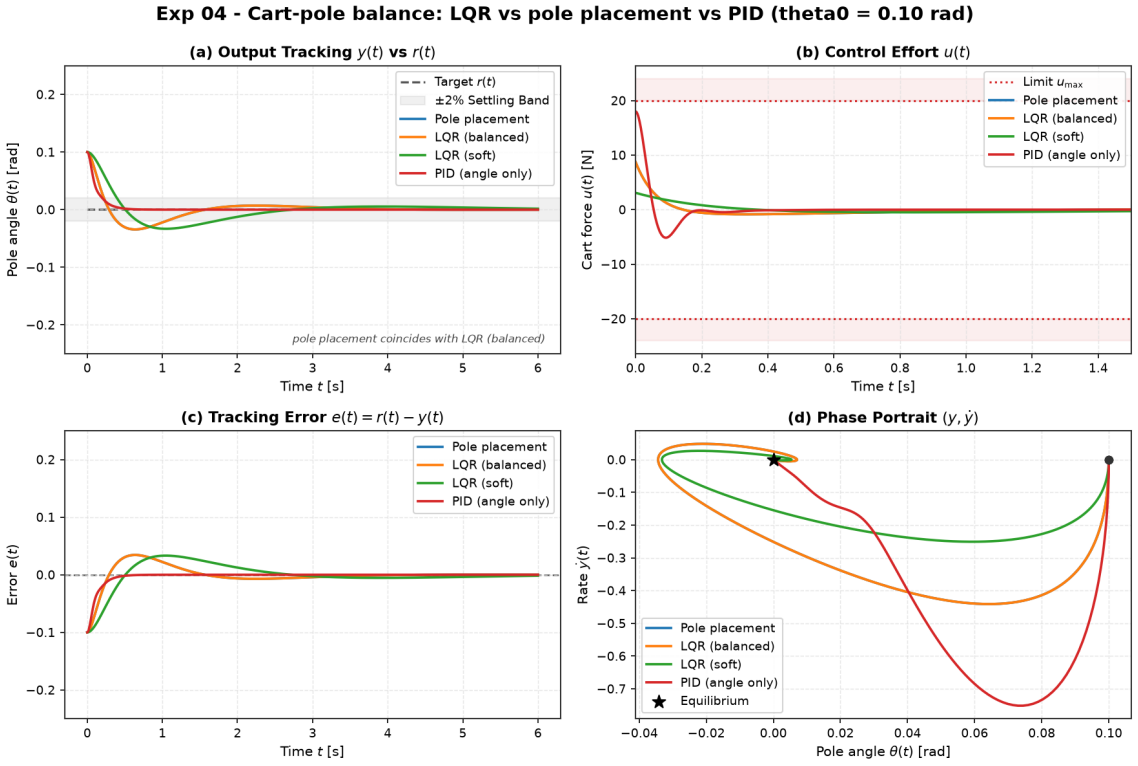

Figure 3: Experiment 04 benchmark comparison on nonlinear Cart-Pole.

6.3 State Estimation: Observers, Kalman Filtering, and Duality

Mathematical Duality: LQR Control vs. Kalman Filtering (LQE).

$$\textbf{LQR Regulator (CARE):} \quad A^\top P + PA - PBR^{-1}B^\top P + Q = 0, \qquad K = R^{-1}B^\top P$$
$$\textbf{LQE Estimator (FARE):} \quad AP + PA^\top - PC^\top R_v^{-1} CP + Q_w = 0, \qquad L = PC^\top R_v^{-1}$$

6.4 Worked Example: LQG on the Cart-Pole (Experiment 06)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^{2}s]	Peak $\lvert u\rvert_{\max}$ [N]	Status
LQR (Full State, Clean)	0.942	0.0150	**1.49**	6.97	Ideal Baseline
LQG (Optimal Filter)	2.070	0.0249	**2.21**	**3.28**	**Optimal Smooth**
Luenberger + K	0.496	0.0165	**16.20**	**15.10**	Severe Noise Chattering
LQG (Overconfident $V/100$)	0.828	0.0215	2.97	4.80	Noise Amplified

Table 4: Benchmark results for Experiment 06 comparing LQG, Luenberger observer, and clean LQR under measurement noise (from `experiments/06_lqg_vs_lqr_measurement_noise/table.md`).

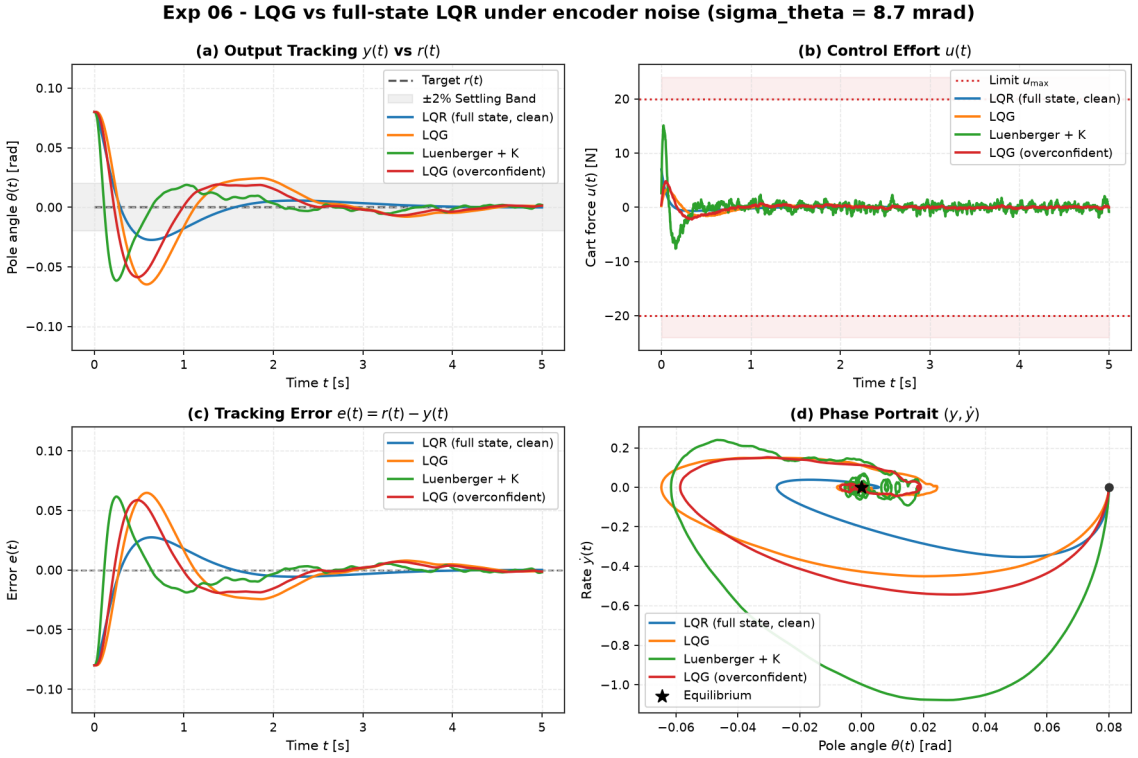

Figure 4: Experiment 06 benchmark comparison under encoder noise.

6.5 Nonlinear Basin of Attraction: Cart-Pole LQR Robustness (Experiment 05)

Tuning	Effort R	Measured θ_0 Edge	Lyapunov Est.	Measured $\dot{\theta}_0$ Edge	Lyapunov Est.
Balanced	0.10	**0.83 rad (48°)**	0.80 rad (46°)	**4.4 rad/s**	4.0 rad/s
Aggressive	0.01	**0.92 rad (53°)**	0.80 rad (46°)	**5.3 rad/s**	4.0 rad/s
Soft	1.00	**1.00 rad (57°)**	0.33 rad (19°)	**5.3 rad/s**	1.3 rad/s

Table 5: Measured recoverable boundaries vs. theoretical Lyapunov sub-level sets on nonlinear Cart-Pole.

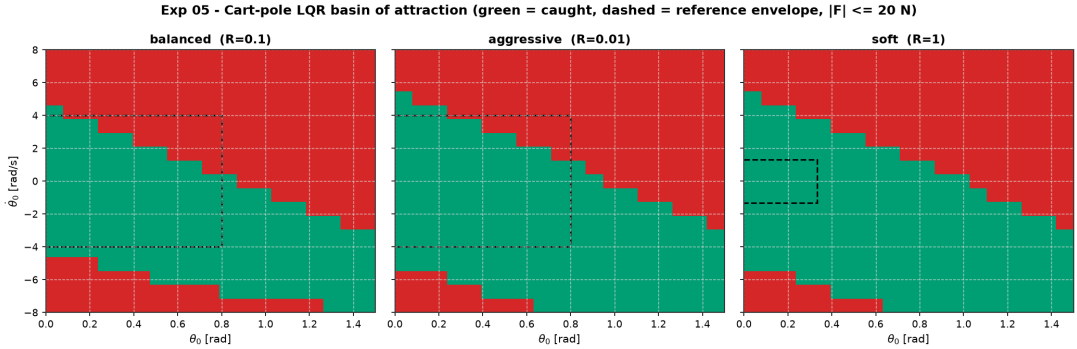

Figure 5: Experiment 05 2D ($\theta_0 \times \dot{\theta}_0$) basin of attraction maps.

6.6 Nonlinear Control & Hybrid Swing-Up: Spong Energy Shaping (Experiment 07)

Energy Gain k_E	Capture t_{cap} [s]	Settle t_s [s]	Switches	Energy E_u [N^{2}s]	Peak $\lvert u\rvert_{\max}$ [N]	Cart Excursion [m]	Status
$k_E = 6$	6.496	7.452	**1**	**52.28**	**12.23**	**1.012**	**Balanced**
$k_E = 10$	2.852	4.058	**1**	87.28	20.00	1.536	**Balanced**
$k_E = 14$	**1.686**	**3.344**	**1**	107.24	19.27	1.940	**Balanced**

Table 6: Benchmark results for Experiment 07 swing-up from $\theta_0 = \pi$ across energy gains $k_E \in \{6, 10, 14\}$ under $|F| \leq 20\,\text{N}$ (from `experiments/07_cartpole_swingup_hybrid/table.md`).

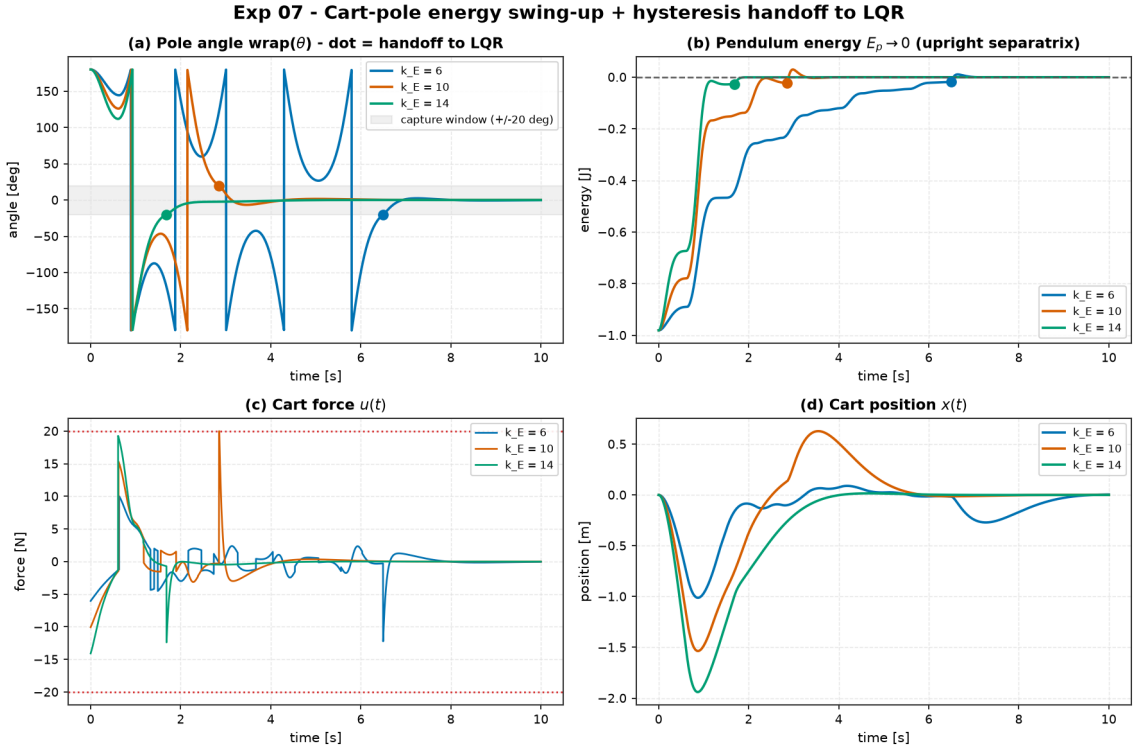

Figure 6: Experiment 07 trajectory traces during swing-up from hanging ($\theta_0 = \pi$).

6.7 System Identification & Data-Driven Control (Experiment 09)

Data Length	Relative Error A	Relative Error K_{id}	RMSE θ [rad]	Stable Runs	Status
300 steps (1 s)	5.86×10^{0}	0.992	26.77	**1 / 6**	Severe Model Drift / Instability
1500 steps (3 s)	4.19×10^{-1}	0.527	0.0336	**4 / 6**	Marginal / Oscillatory
12000 steps (24 s)	$\mathbf{2.17 \times 10^{-1}}$	**0.493**	**0.0246**	**6 / 6 (100%)**	**Consistently Stable**

Table 7: Experiment 09 Monte Carlo evaluation (6 seeds) of LQR designed on identified models (from `experiments/09_control_on_identified_model/table.md`).

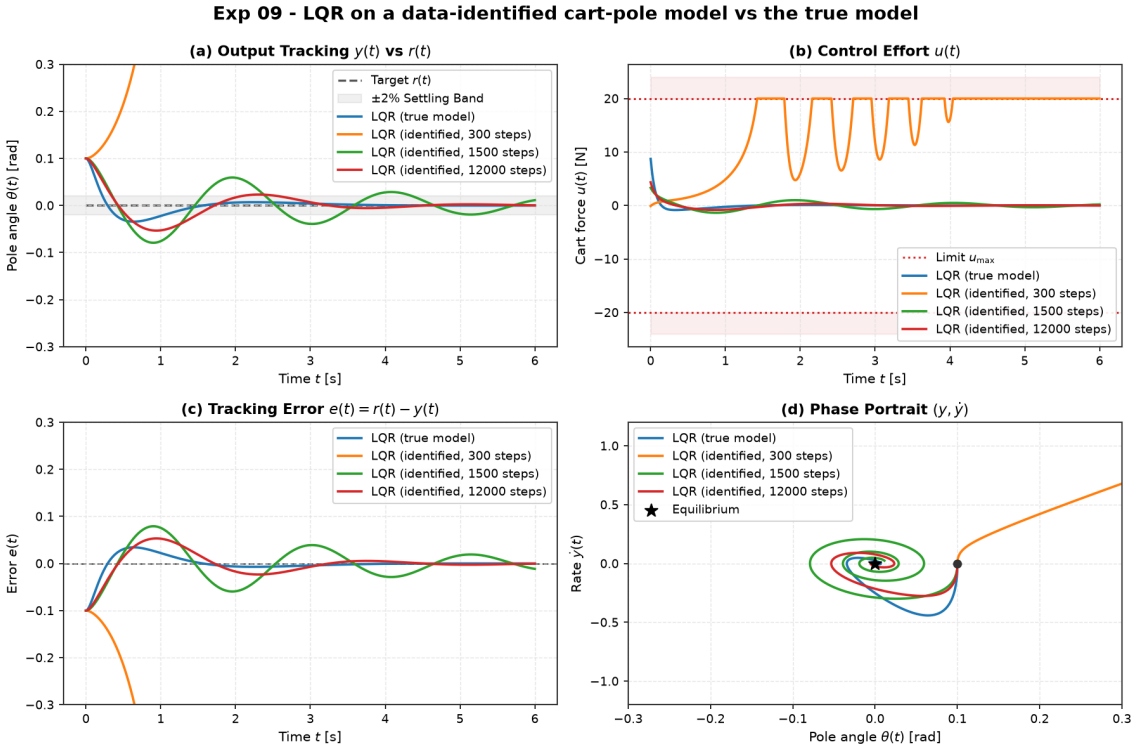

Figure 7: Experiment 09 benchmark comparison showing LQR performance designed on identified models.

6.8 Constrained Model Predictive Control (Experiment 08)

Controller	Cart Peak $\lvert x\rvert_{\max}$ [m]	Constraint Violation [m]	Settle θ t_s [s]	Energy E_u [N^{2}s]	Status
LQR	0.6162	**0.1162 (23.2%)**	2.79	**28.8**	Rail Violation
MPC (Unconstrained)	0.6232	**0.1232 (24.6%)**	2.81	**27.7**	Rail Violation
MPC ($\lvert x_{\text{cart}}\rvert \leq 0.5$ m)	**0.5014**	**0.0014 (0.28%)**	**2.56**	46.3	**Strictly Compliant**

Table 8: Benchmark results for Experiment 08 on nonlinear Cart-Pole with $|F| \leq 20\,\text{N}$ and rail limit $|x| \leq 0.5\,\text{m}$ from $\theta_0 = 0.35\,\text{rad}$ (from `experiments/08_mpc_vs_lqr_constrained_cartpole/table.md`).

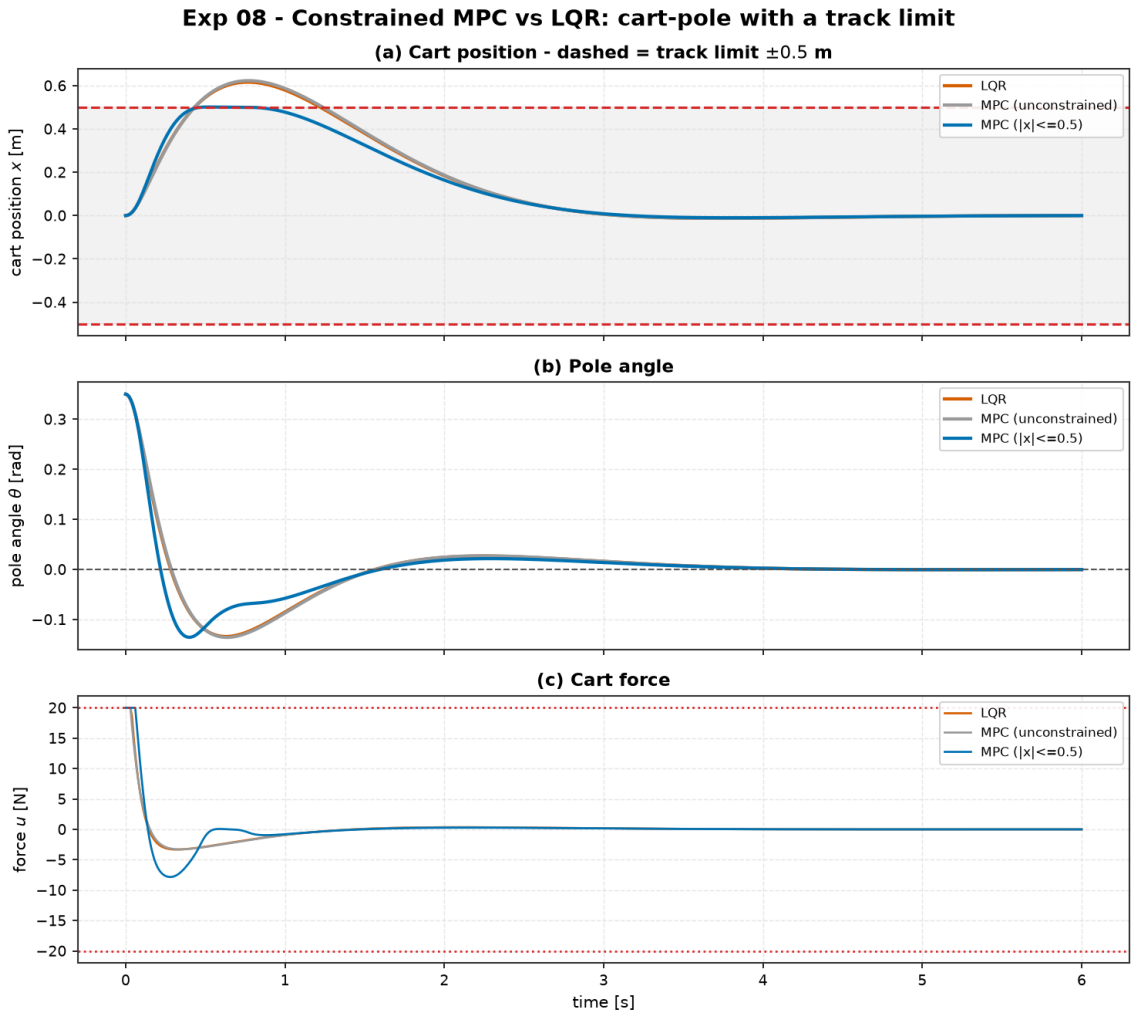

Figure 8: Experiment 08 trajectory traces highlighting Cart Position $x(t)$ with the $\pm 0.5\,\mathrm{m}$ track limit.

7 Theory: Machine Learning and Learned Dynamics (Module 06)

7.1 Worked Example: Planning with Learned Dynamics vs. True Model (Experiment 10)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^{2}s]	Peak $\lvert u\rvert_{\max}$ [N]	Status
LQR (True Model)	1.26	**0.0374**	**8.42**	17.40	Analytic Optimum
SamplingMPC (True Model)	4.44	0.0448	10.80	6.59	Stable (CEM True)
SamplingMPC (Learned Model)	**3.56**	**0.0462**	**10.20**	**6.00**	**Stable (CEM Learned)**

Table 9: Benchmark results for Experiment 10 comparing CEM-MPC on learned neural dynamics vs. true model and LQR (from `experiments/10_planning_learned_vs_true_model/table.md`).

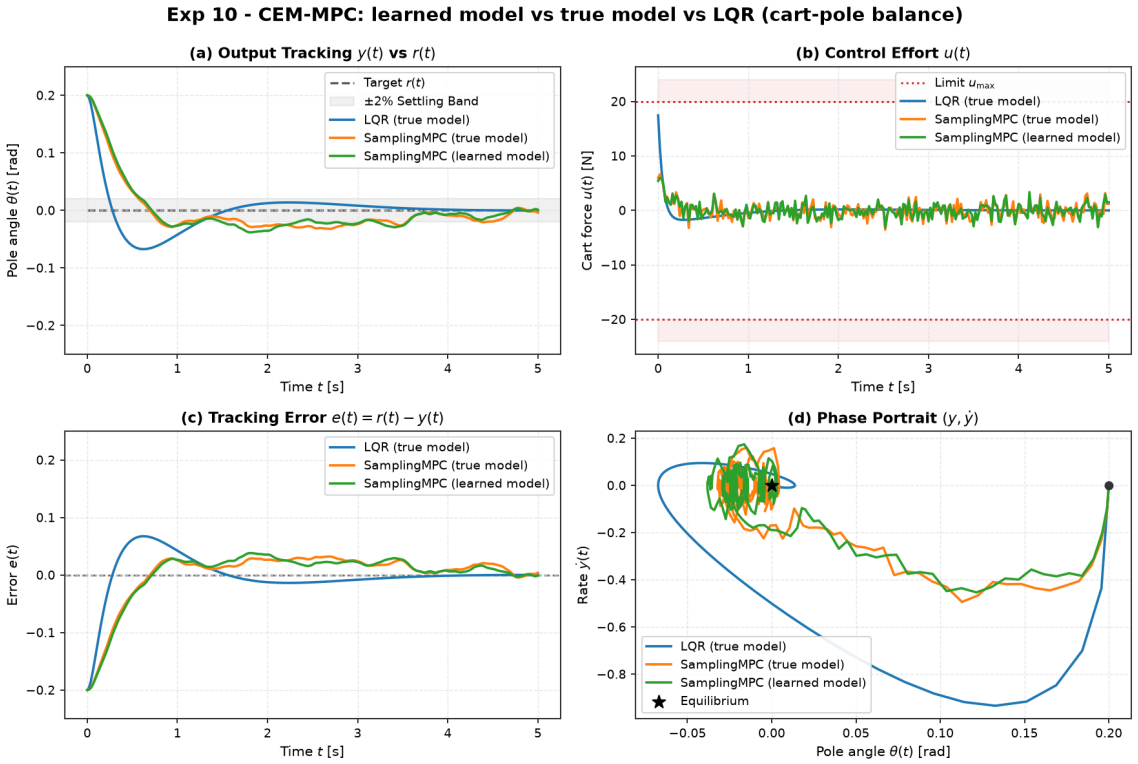

Figure 9: Experiment 10 benchmark comparison: SamplingMPC on Learned Neural Model vs. True Model.

8 Theory: Reinforcement Learning for Dynamical Systems (Module 07)

8.1 Worked Example: Tabular Q-Learning vs. Classical Control on Pendulum (Experiment 11)

Controller	Task	Min Err [rad]	Held Upright?	t_{upright} [s]	Final Err [rad]	Energy E_u	Train Samples	Parameters
Energy-Shaping + LQR	Swing-Up	**0.00**	**Yes**	**4.00**	**0.000**	**45.9**	**0**	**3**
Tabular Q-Learning	Swing-Up	0.02	No	9.10	1.479	101.2	450,000	61,875
LQR	Balance	**0.00**	**Yes**	**0.85**	**0.000**	**11.7**	**0**	**2**
Tabular Q-Learning	Balance	0.05	No	3.45	1.527	40.4	180,000	51,975

Table 10: Benchmark results for Experiment 11 comparing Tabular Q-Learning against Classical Energy-Shaping and LQR on the Inverted Pendulum (from `experiments/11_qlearning_vs_classical/table.md`).

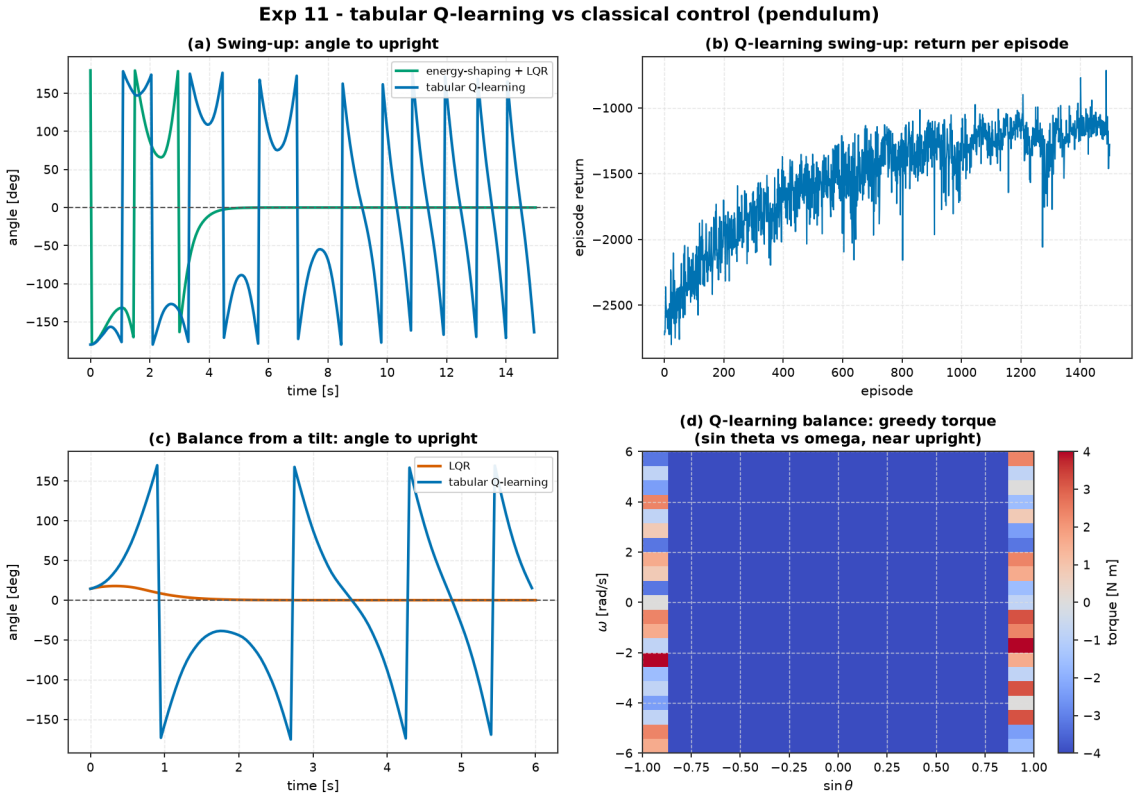

Figure 10: Experiment 11 benchmark comparison: Tabular Q-Learning vs. Classical Control on Inverted Pendulum.

8.2 Worked Example: The RL Zoo vs. LQR on Cart-Pole Balance (Experiment 18)

Agent / Controller	Greedy Return	Held Steps (/200)	Env Steps	Train Time [s]	Parameters
LQR (Analytic CARE)	**-0.3**	**200.0**	**0**	**0.0**	**4**
Tabular Q-Learning	-55.5	92.0	300,000	7.1	354,375
DQN (From Scratch)	-2.6	**200.0**	**90,177**	68.7	4,805
PPO (From Scratch)	**-0.3**	**200.0**	240,000	72.6	4,545
PPO (Stable-Baselines3)	-44.9	52.5	120,000	129.6	9,091

Table 11: Benchmark results for Experiment 18 comparing the RL zoo against LQR on Cart-Pole balance (from `experiments/18_rl_zoo_vs_lqr/table.md`).

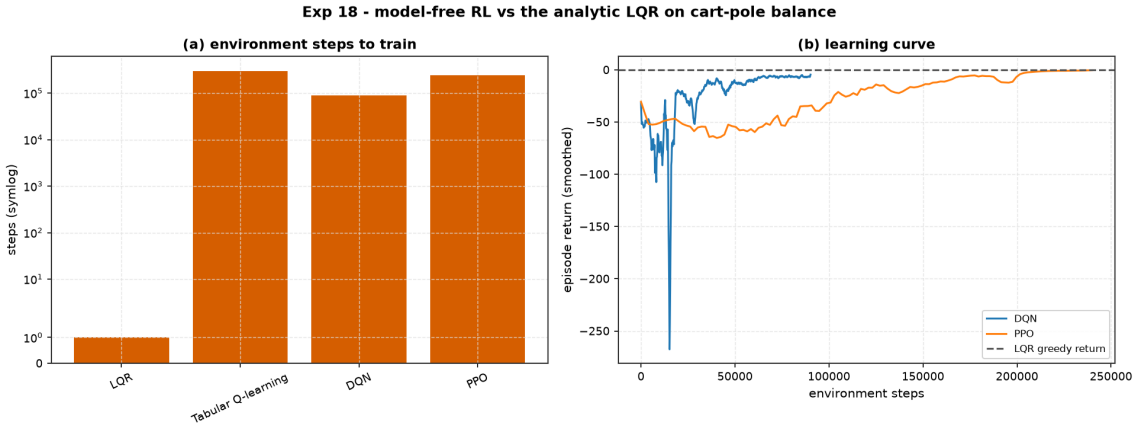

Figure 11: Experiment 18 benchmark comparison: RL Zoo vs. LQR on Cart-Pole balance.

9 Theory: AI + Control Hybrid Architectures (Module 08)

9.1 Worked Example: Shielded Tabular Q-Learning on Pendulum (Experiment 12)

Controller	Min Err [rad]	Final Err [rad]	Held Upright?	t_{upright} [s]	Energy E_u	Shield Active
Tabular Q-Learning (Raw)	0.028	1.406	No	10.30	105.4	0.0%
Shielded (RL + Classical Finisher)	**0.000**	**0.000**	**Yes**	**9.30**	**68.0**	**43.7%**

Table 12: Benchmark results for Experiment 12 comparing Raw Tabular Q-Learning vs. Shielded Hybrid Control on the Inverted Pendulum (from `experiments/12_shielded_qlearning/table.md`).

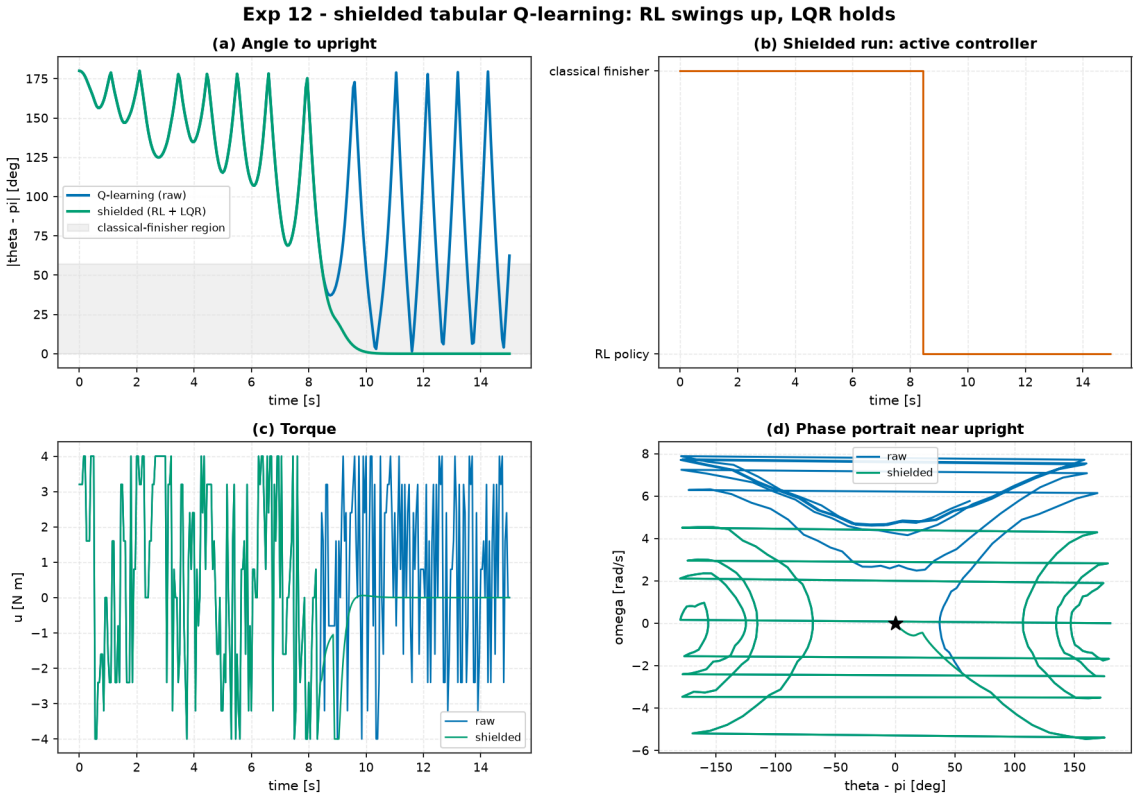

Figure 12: Experiment 12 benchmark comparison: Raw Tabular Q-Learning vs. Shielded Hybrid Control on Inverted Pendulum.

10 Theory: Advanced State Estimation, Adaptive Control, and Robotics Capstones (Module 09)

10.1 Worked Example: Crazyflie 2.0 Flying a Lemniscate (Experiment 14)

Controller	RMS Pos Err [mm]	Max Pos Err [mm]	RMS Pitch [°]	Control Energy $\int \|\Delta u\|^2$	Thrust Saturation
LQR + Flatness Feedforward	**43.5**	112.0	4.3°	0.033	0.7%
LQR Feedback Only	51.0	**109.0**	4.4°	0.038	0.7%
Linear MPC (Single Setpoint)	142.0	208.0	4.3°	**0.026**	**0.1%**
Linear MPC (Reference Preview)	**47.9**	134.0	4.5°	0.033	0.6%

Table 13: Benchmark results for Experiment 14: Crazyflie 2.0 figure-8 tracking under 11% wind gust (from `experiments/14_quadrotor_figure8_tracking/table.md`).

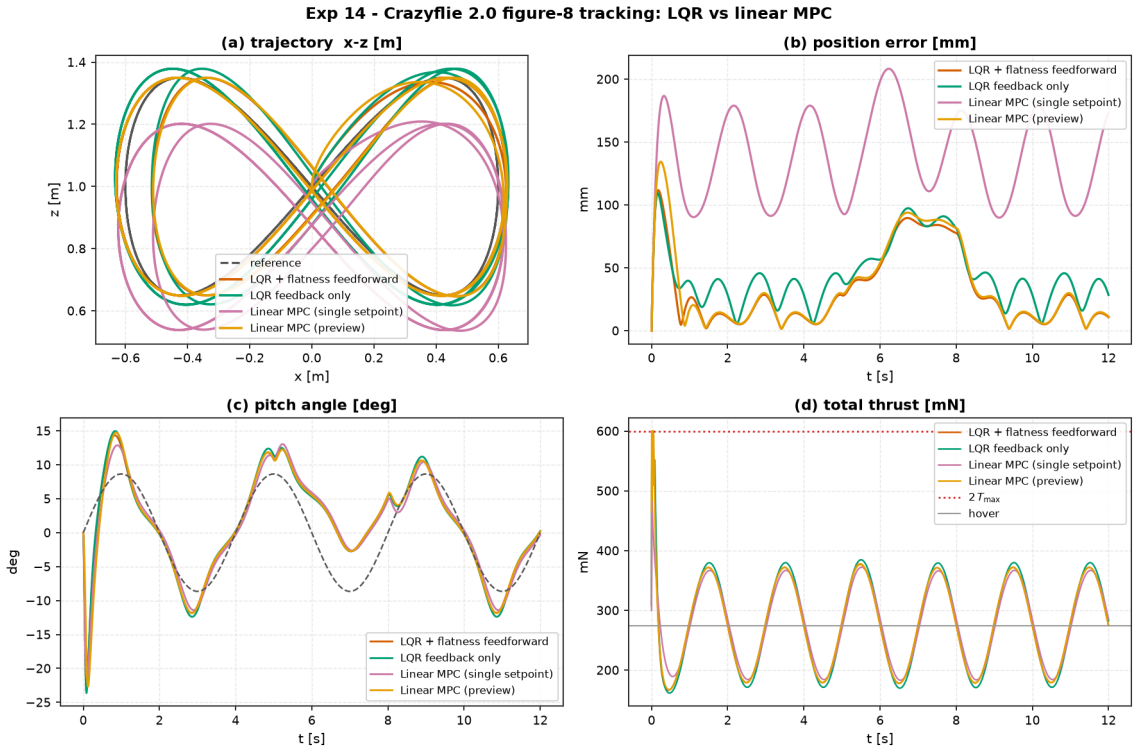

Figure 13: Experiment 14 benchmark comparison: Crazyflie 2.0 planar quadrotor flying a lemniscate figure-8 through a sustained wind gust.

10.2 Worked Example: EKF Output Feedback under Noisy Partial Sensing (Experiment 15)

Controller	RMS Pos Err [mm]	Max Pos Err [mm]	RMS Pitch [°]	Control Energy $\int \|\Delta u\|^2$	RMS Vel Err [m/s]
LQR (Clean Full State)	**9.04**	**49.0**	1.26°	**0.0024**	—
LQR + EKF (Nonlinear Observer)	**9.51**	50.6	**1.25°**	**0.0026**	**0.008**
LQR + Finite-Difference Vel	18.32	47.4	1.89°	**0.3580 (150×)**	0.332 (40×)

Table 14: Benchmark results for Experiment 15: Crazyflie 2.0 output feedback tracking with EKF vs. naive differencing (from `experiments/15_quadrotor_ekf_output_feedback/table.md`).

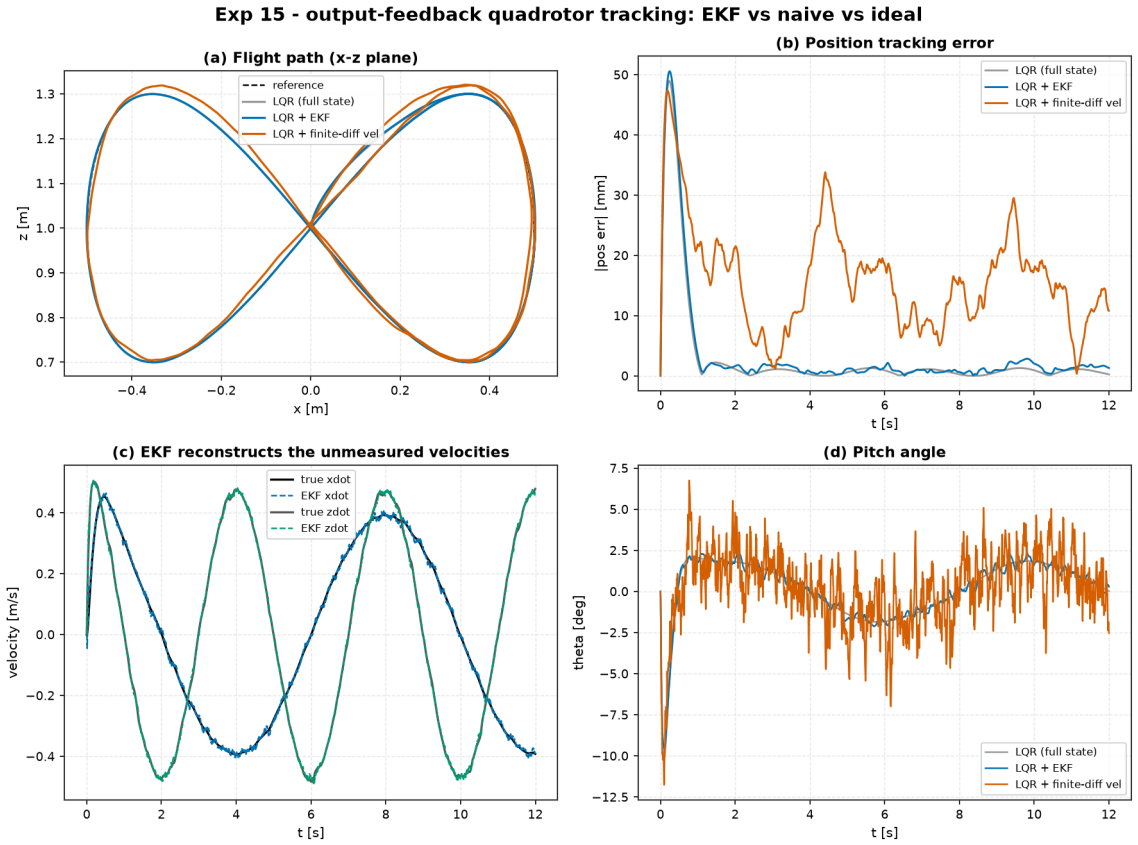

Figure 14: Experiment 15 benchmark comparison: EKF Output-Feedback Quadrotor Tracking.

10.3 Worked Example: EKF vs. UKF on Nonlinear Pendulum Estimation (Experiment 16)

Regime	Filter	RMS Error [rad]	Final Error [rad]	Recovered?
Hard (sin/cos, π-off init)	**EKF**	6.514	**6.282** ($\approx 2\pi$)	**False (Trapped)**
Hard (sin/cos, π-off init)	**UKF**	**1.382**	**0.068**	**True (Converged)**
Mild (sin/rate, good init)	**EKF**	**0.021**	**0.026**	**True**
Mild (sin/rate, good init)	**UKF**	**0.021**	**0.026**	**True**

Table 15: Benchmark results for Experiment 16: EKF vs. UKF on nonlinear Pendulum estimation (from `experiments/16_ekf_vs_ukf/table.md`).

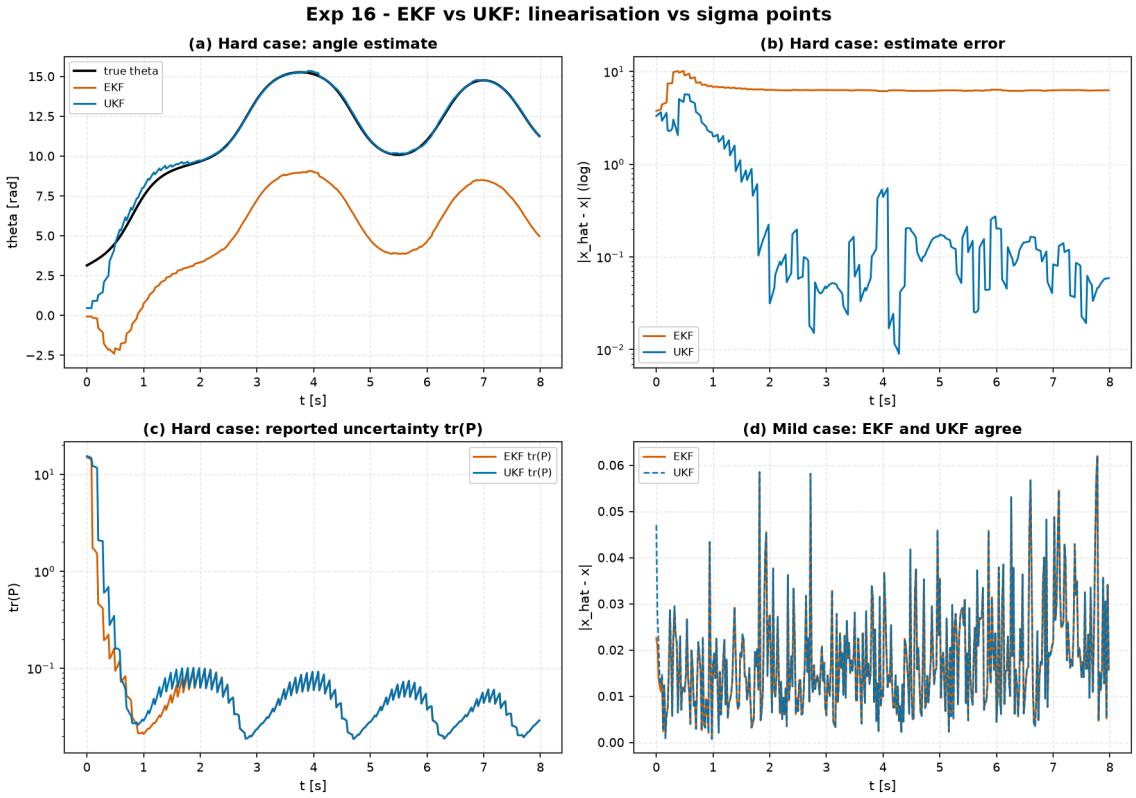

Figure 15: Experiment 16 benchmark comparison: EKF vs. UKF on nonlinear Pendulum estimation.

10.4 Worked Example: Adaptive vs. Fixed Control on a Drifting Plant (Experiment 17)

Controller	RMS Err (Early $t \leq 5$s)	RMS Err (Late $t \geq 15$s)	Final Settle Err [m]	Control Energy E_u
LQR (Nominal $k = 1$)	0.2573	0.4013	0.4217	180.7
LQR (Worst-Case $k = 5$)	0.3577	0.5194	0.5406	**123.3**
MRAC (Adaptive)	**0.0008**	**0.0008**	**0.0008**	413.5
GainScheduled LQR	0.2916	0.5075	0.5396	142.3

Table 16: Benchmark results for Experiment 17: Adaptive MRAC vs. Fixed LQR on a drifting plant (from `experiments/17_adaptive_vs_fixed_changing_plant/table.md`).

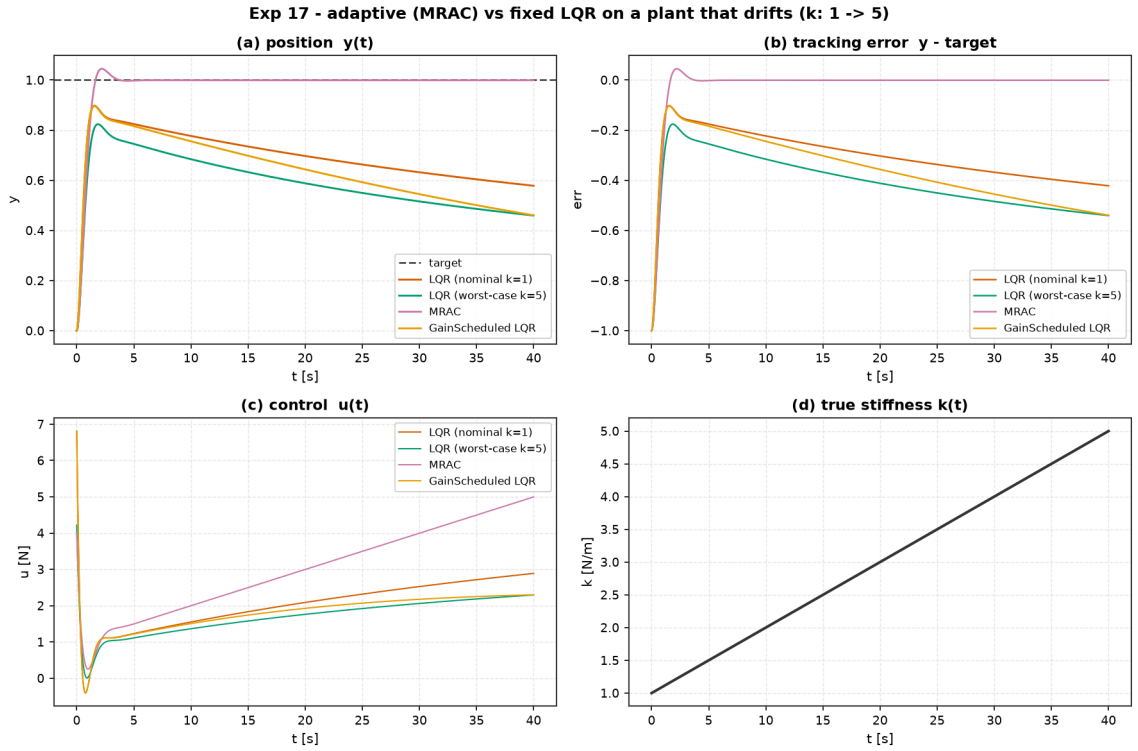

Figure 16: Experiment 17 benchmark comparison: Adaptive MRAC vs. Fixed LQR on drifting spring plant.

10.5 Worked Example: Obstacle-Aware Nonlinear MPC on the Quadrotor (Experiment 20)

Controller	RMS Pos Err [mm]	Min Clearance [mm]	Steps in Keep-Out	Control Energy E_u
LQR + Flatness Feedforward	**86.26**	**-85.80**	**26 (Violates)**	0.4804
SamplingMPC (True Dynamics)	234.4	**+10.88**	**0 (Compliant)**	**0.0329**
SamplingMPC (Learned Grey-Box Model)	254.6	**+26.06**	**0 (Compliant)**	0.0343

Table 17: Benchmark results for Experiment 20: Obstacle-aware nonlinear MPC on Crazyflie 2.0 (from `experiments/20_quadrotor_obstacle_nmpc/table.md`).

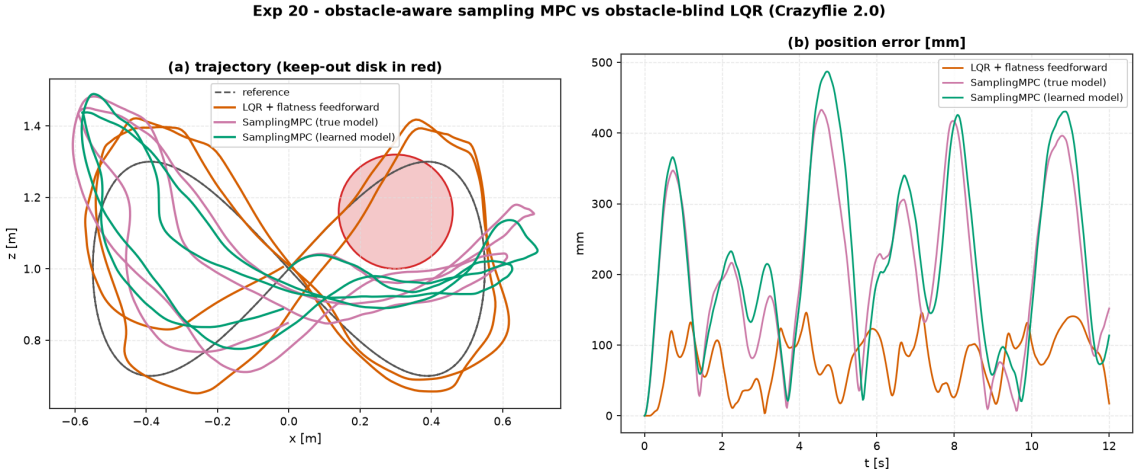

Figure 17: Experiment 20 benchmark comparison: Obstacle-aware nonlinear MPC on Crazyflie 2.0 quadrotor.

10.6 Worked Example: The Grand Capstone Bake-Off (Experiment 21)

Experiment 21 conducts the ultimate five-way capstone bake-off on the full Crazyflie 2.0 figure-8 course combined with a keep-out obstacle disk ($r = 0.16\,\text{m}$ at $(0.30, 1.16)\,\text{m}$) and a 30 mN lateral wind gust during $t \in [4.5, 7.5]\,\text{s}$. We evaluate every major paradigm under the standardized multi-objective scoring rubric (tracking RMSE 0.40, energy 0.20, slew 0.10, safety compliance 0.15, wind robustness 0.15):

Controller Paradigm	RMS [mm]	Robust RMS [mm]	Energy E_u	Slew	Obstacle Steps	Thrust Sat.	Latency [ms]	Score
LQR + Flatness	99.9	102.0	0.483	378.0	**28 (DQ)**	**72.4%**	**0.05**	**0.0**
Linear MPC (Preview)	243.0	242.0	**0.0034**	**0.0639**	**20 (DQ)**	0.0%	11.5	**0.0**
Sampling MPC (Learned)	316.0	275.0	0.0334	14.3	**0 (Clean)**	0.0%	38.4	**8.0 (PASS)**
Imitation Policy (BC+PPO)	**41.4**	**35.7**	0.0040	0.0580	**46 (DQ)**	0.0%	0.09	**0.0**
Imitation + MPC Shield	317.0	283.0	0.0316	12.8	**0 (Clean)**	0.0%	38.4	**7.9 (PASS)**

Table 18: Benchmark results for Experiment 21: The Grand Capstone Bake-Off on Crazyflie 2.0 (from `experiments/21_grand_capstone_bakeoff/table.md`).

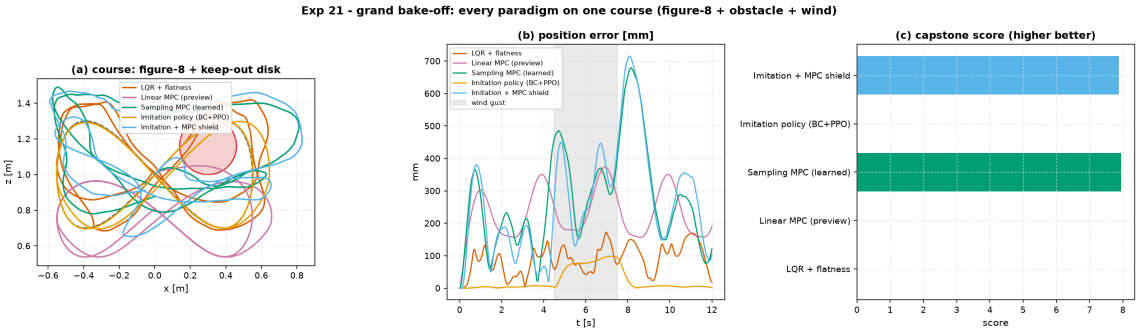

Figure 18: Experiment 21 Grand Capstone Bake-Off on Crazyflie 2.0 quadrotor: Course layout, position error over time, and multi-objective score breakdown.

11 Theory: The Intelligent Control Challenge (ICC) Benchmark (Module 10)

11.1 Worked Example: Multi-Paradigm Challenge Leaderboard (Experiment 19)

Submission Paradigm	Track 1 (MSD)	Track 1 (DC Motor)	Track 2 (Pendulum)	Track 2 (Cart-Pole)	Track 3 (Robustness)
Linear MPC	**21.1 (PASS)**	**41.3 (PASS)**	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
LQR	7.4 (PASS)	36.8 (PASS)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
Energy+LQR Hybrid	1.8 (PASS)	0.0 (FAILED)	**23.8 (PASS)**	**9.6 (PASS)**	**29.9 (PASS)**
PID	0.6 (PASS)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
Tabular Q-Learning	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)

Table 19: Experiment 19 Intelligent Control Challenge Leaderboard: Composite Score / 100 (from `experiments/19_icc_leaderboard/leaderboard.md`).

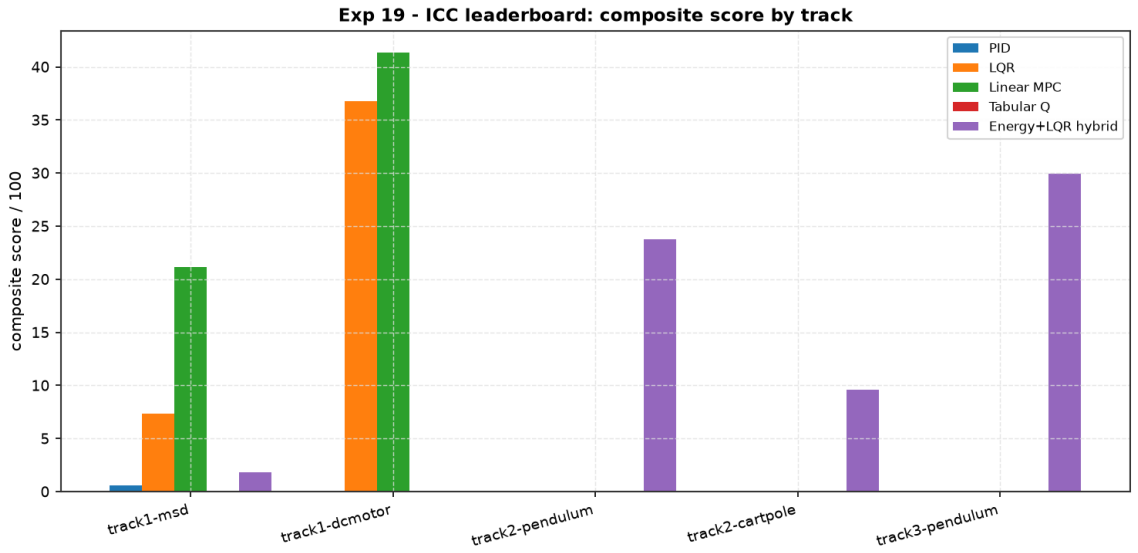

Figure 19: Experiment 19 Intelligent Control Challenge Leaderboard: Composite score comparison across all five paradigms and benchmark tracks.

12 Phase 2: Real Multi-Body Systems and Method Depth

Phase 2 deepens the investigation across two major axes: extending the library to multi-body robotic systems with kinematic constraints (mobile robotics, robot arms) and replacing stochastic sampling planners with gradient-based real-time iteration Nonlinear MPC (iLQR / RTI-NMPC).

12.1 Worked Example: Differential-Drive Robot Path Following (Experiment 22)

We evaluate four controllers on a non-holonomic differential-drive robot (state $x = [p_x, p_y, \theta, v, \omega]^\top$ with actuator first-order lag time constant $\tau = 50\,\text{ms}$) tracking a 5-waypoint cubic spline at a 0.15 m/s cruise speed:

Controller	RMS Pos Err [mm]	Max Pos Err [mm]	RMS Cross-Track [mm]	Completion %	Control Energy	Status
Pure Pursuit	**69.51**	**107.4**	34.57	**98.39%**	**1.949**	Corner Cutting
Stanley	111.4	257.0	9.75	95.51%	2.412	Schedule Lag
Path LQR	96.45	229.6	**9.25**	96.15%	2.278	**Optimal Balance**
Kinematic MPC	128.0	290.4	19.16	94.88%	2.366	Unconstrained

Table 20: Benchmark results for Experiment 22: Differential-drive mobile robot path following (from `experiments/22_diffdrive_path_following/tracking.md`).

Key Takeaways:

1. **Pure Pursuit chases the point, not the path:** Achieves the lowest along-track error (69.5 mm) and highest completion (98.4%), but geometrically cuts corners on curves, producing a 35 mm cross-track offset.
2. **Stanley hugs the geometry:** Drives cross-track error to $< 10\,\text{mm}$, but lacks schedule anticipation, resulting in along-track phase lag.
3. **Path LQR provides the ideal synthesis:** Path curvature feedforward ($\omega_{\text{ff}} = \kappa v$) plus linearised error feedback delivers the tightest cross-track error (9.25 mm) with low control energy.

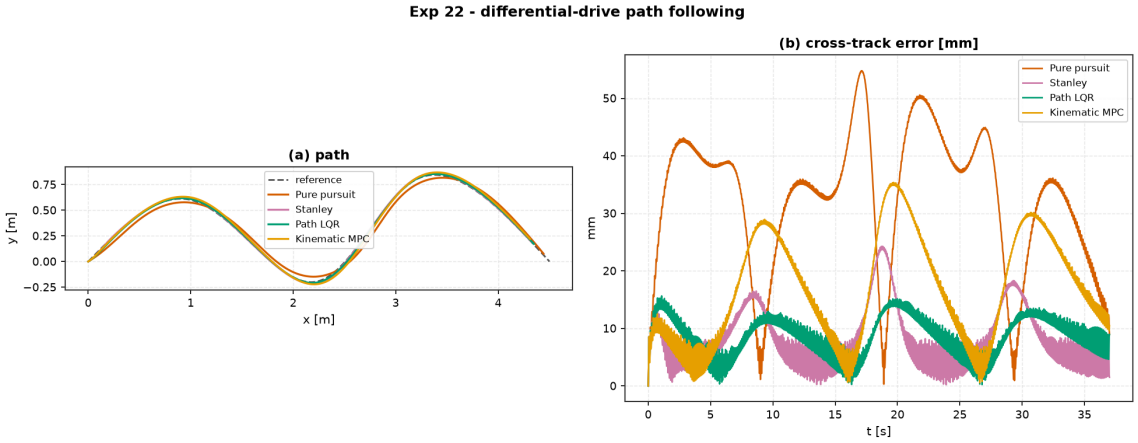

Figure 20: Experiment 22 differential-drive robot path following: Waypoint trajectory and cross-track error comparison.

12.2 Worked Example: Two-Link Planar Arm Tracking & Payload Adaptation (Experiment 23)

We evaluate joint trajectory tracking on a 2-link planar manipulator arm governed by Euler–Lagrange equations $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$. In Experiment 23A (nominal model), model-based computed torque and joint LQR achieve millimeter-precision tracking. In Experiment 23B, an unknown 0.5 kg point mass payload is stepped onto the wrist:

Controller (Nominal Tracking)	RMS Err [mm]	Max Err [mm]	Cross-Track [mm]	Completion %	Energy
PD + Gravity Comp	32.24	54.47	21.43	99.38%	891.2
Computed Torque	4.322	29.59	**3.370**	**100.0%**	**164.3**
Joint LQR	**2.572**	**12.02**	4.143	**100.0%**	164.5
Joint MPC	9.999	50.50	3.383	**100.0%**	164.4
Controller (0.5 kg Wrist Payload)	**RMS Err [mm]**	**Max Err [mm]**	**Cross-Track [mm]**	**Completion %**	**Energy**
Computed Torque (Nominal Model)	**393.8**	**519.5**	**262.3**	**36.9% (FAILED)**	492.8
Adaptive Computed Torque (Slotine–Li)	**4.927**	**10.16**	**5.199**	**100.0% (PASS)**	763.5

Table 21: Benchmark results for Experiment 23: Two-link arm joint tracking under nominal and unknown payload step conditions (from `experiments/23_twolink_arm_tracking/`).

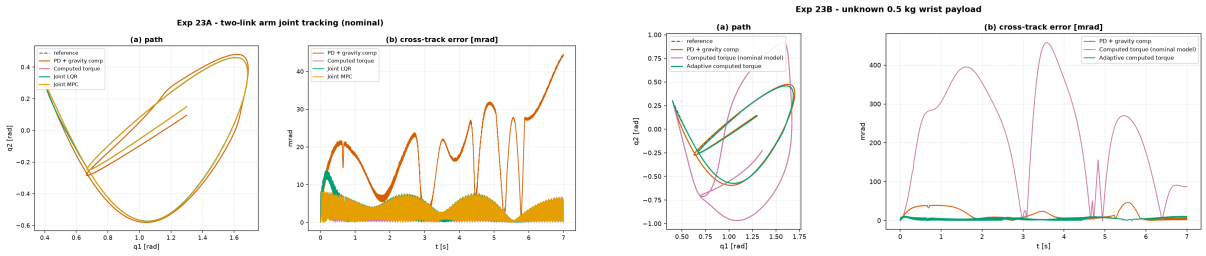

Figure 21: Experiment 23 two-link planar arm: (Left) Nominal joint-space tracking; (Right) Slotine–Li adaptive compensation recovering from a 0.5 kg wrist payload step.

12.3 Worked Example: iLQR / Real-Time Iteration NMPC vs. Sampling MPC (Experiment 24)

Experiment 24 addresses the central computational bottleneck uncovered in the Capstone Bake-Off: stochastic Cross-Entropy Method (CEM) sampling MPC is computationally heavy (38 ms)

and tracks loosely. We compare `aimct.controllers.ILQR` (gradient-based iLQR with backward Riccati passes and Real-Time Iteration NMPC) against `SamplingMPC` on two benchmark tasks:

Task 1: Cart-Pole Swing-Up	Time Upright [s]	Final Angle [°]	Peak Force [N]	Median Latency [ms]	P95 Latency [ms]
Sampling MPC (CEM)	1.60	1.59°	17.20	97.95	116.0
iLQR / RTI-NMPC	**1.12**	**0.39°**	20.00	**28.79 (3.4× faster)**	**35.34**
Task 2: Quadrotor Figure-8	**RMS Pos Err [mm]**	**Max Pos Err [mm]**	**Control Energy**	**Median Latency [ms]**	**Real-Time Budget**
Sampling MPC (CEM)	202.2	439.1	0.0298	26.58	Exceeds 20 ms
iLQR / RTI-NMPC	**1.34 (150× tighter)**	**2.40**	**0.0041**	**14.59**	**Complies (< 20 ms)**

Table 22: Benchmark results for Experiment 24: Gradient iLQR / RTI-NMPC vs. Stochastic Sampling MPC (from `experiments/24_ilqr_vs_sampling_mpc/table.md`).

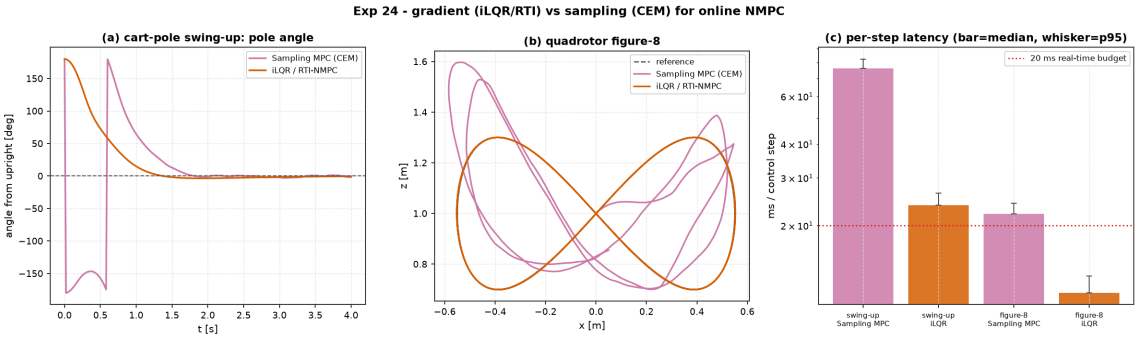

Figure 22: Experiment 24 benchmark comparison: iLQR / RTI-NMPC vs. Sampling MPC on Cart-Pole swing-up and Crazyflie 2.0 Quadrotor Figure-8 tracking.

13 Engineering Synthesis & Decision Guide

The central mission of this project was never to demonstrate that a neural network can steer a dynamical system, but to build **grounded engineering judgment**: given physical dynamics, constraints, sensor suites, and operational requirements, which method does empirical evidence justify?

Operational Situation	Recommended Methodology	Empirical Evidence
Equations of motion known / linearisable	**LQR / Pole Placement**, then **Linear MPC** under constraints	Exp 04, Exp 08
Hard state/actuator inequality bounds	**Linear MPC** (active-set condensed QP)	Exp 08, Exp 20
Nonlinear dynamics with real-time deadline	**iLQR / RTI-NMPC** (1.3 mm error, 14.6 ms solve)	Exp 24
Multi-body robot arm trajectory tracking	**Computed Torque** / **Joint LQR**; **Slotine–Li MRAC** under payload	Exp 23
Wheeled mobile robot path following	**Path LQR** (curvature feedforward) / **Stanley**	Exp 22
Smooth reference trajectory tracking	**LQR + Flatness Feedforward** or **Preview MPC**	Exp 14
Constant disturbances / steady-state error	**Integral Action (LQI)** or **MRAC**	Exp 03, Exp 17, Live Sandbox
Plant parameters drift over time (wear/load)	**MRAC** (matched) / **Gain Scheduling** (measured)	Exp 17, Exp 23
Noisy / partial sensor measurements	**Kalman Filter (LQG)**; **EKF / UKF** if nonlinear	Exp 06, Exp 15, Exp 16
No equations, but rollout data available	**Least-Squares / DMDc SysID** → Model-based design	Exp 09
No model, nonlinear, simulator available	**Sampling MPC (CEM)** over learned residual model	Exp 10, Exp 20
Black-box interaction only (no physics model)	**Deep RL (DQN / PPO)** — anticipate high sample bill	Exp 11, Exp 18
Deploying uncertified learned policies	**Safety Shield** with certified classical fallback	Exp 12, Exp 21

Table 23: Cross-experiment engineering decision matrix (from `docs/DECISION-GUIDE.md`).

13.1 Recurring Engineering Lessons Across 24 Experiments

Across 24 empirical benchmarks, six fundamental principles recur consistently:

1. **Feedforward Does the Heavy Lifting:** Model-based feedforward (differential flatness, steady-state inversion, curvature feedforward, reference preview) generates the baseline control trajectory directly, leaving feedback to correct only small modeling residuals and unexpected disturbances (Exp 03, 14, 17, 22).

2. **Integral Action is Mandatory for Constant Disturbances:** No amount of static state-feedback tuning (K) eliminates steady-state droop under persistent external loads (wind, gravity bias); explicit integrator augmentation or parameter adaptation is required (Exp 03, 17, 23).

3. **Cost and Weight Scaling is Load-Bearing:** On ill-conditioned or multi-timescale physical systems (e.g., quadrotor position vs. moment of inertia), naive identity weights ($Q = I, R = I$) fail. Bryson-rule normalization ($Q_{ii} = 1/x_{i,\max}^2, R_{jj} = 1/u_{j,\max}^2$) is essential for well-behaved optimal gains (Exp 14, 18, 19).

4. **Model Error is Survivable within Robustness Margins:** Full-state LQR provides infinite upper gain margin and $[-6\,\text{dB}, +\infty\,\text{dB}]$ lower gain margin, tolerating up to 50% parameter identification error without loss of asymptotic stability (Exp 09).

5. **Gradient-Based Trajectory Optimization Beats Stochastic Sampling:** iLQR / Real-Time Iteration NMPC converges to true Karush–Kuhn–Tucker (KKT) stationarity in 14.6 ms with 1.3 mm tracking precision, outperforming loose Monte Carlo sampling (202 mm error at 26.6 ms) (Exp 24).

6. **Reporting Failure Modes is as Essential as Reporting Wins:** Engineering judgment relies on knowing exact breaking points: linearisation breakdown past 23° (Exp 02), EKF false basin trapping (Exp 16), closed-loop SysID correlation bias (Exp 09), nominal computed torque collapse under payload step (Exp 23), and pure RL bootstrap failure on low-inertia plants (Exp 21).

14 Project Status and Roadmap

Phase 1: Foundation, Curriculum & Capstone — Completed. All curriculum modules, capstone robotics showcases, deep RL agents, adaptive controllers, and benchmark challenge suites are fully implemented from scratch, tested, and validated across 21 experiments (Modules 01–10).

Phase 2: Post-Capstone Depth, Real Systems & Packaging — In Progress.

- **Track A (Real Multi-Body Systems):** Differential-drive mobile robot (Exp 22: path tracking), 2-link planar manipulator arm (Exp 23: computed torque vs. Slotine–Li MRAC with payload step), and bicycle-model ground vehicle.
- **Track B (Algorithmic Depth):** Real-time iteration Nonlinear MPC (Exp 24: iLQR / RTI-NMPC vs. Sampling MPC), Soft Actor-Critic (SAC) continuous RL, direct collocation trajectory optimization, and formal Behavioral Cloning + DAgger.
- **Track C (Harness Infrastructure):** Standardized trajectory-tracking benchmark suite (`aimct.benchmarks.track_trajectory`) with cross-track / along-track error metrics, and `aimct.trajectories` reference generator library.
- **Track D (PyPI Distribution):** Production packaging under `aimct`, console entry points, automated GitHub Action release pipeline with PyPI OIDC Trusted Publishing, and release runbook.

CI passes on Python 3.10–3.13 with **373 passing unit tests**.

Reproduction Commands.

```
git clone https://github.com/zalihthomas-ui/ai-meets-control-theory.git
cd ai-meets-control-theory
pip install -e ".[dev,ml,xcheck]"
python -m pytest -q
python experiments/03_pid_stabilizes_unstable/run.py
python experiments/04_lqr_vs_pole_placement_cartpole/run.py
python experiments/05_cartpole_basin_of_attraction/run.py
python experiments/06_lqg_vs_lqr_measurement_noise/run.py
python experiments/07_cartpole_swingup_hybrid/run.py
python experiments/08_mpc_vs_lqr_constrained_cartpole/run.py
python experiments/09_control_on_identified_model/run.py
python experiments/10_planning_learned_vs_true_model/run.py
python experiments/11_qlearning_vs_classical/run.py
python experiments/12_shielded_qlearning/run.py
python experiments/14_quadrotor_figure8_tracking/run.py
python experiments/15_quadrotor_ekf_output_feedback/run.py
python experiments/16_ekf_vs_ukf/run.py
python experiments/17_adaptive_vs_fixed_changing_plant/run.py
python experiments/18_rl_zoo_vs_lqr/run.py
python experiments/19_icc_leaderboard/run.py
python experiments/20_quadrotor_obstacle_nmpc/run.py
python experiments/21_grand_capstone_bakeoff/run.py
python experiments/22_diffdrive_path_following/run.py
python experiments/23_twolink_arm_tracking/run.py
python experiments/24_ilqr_vs_sampling_mpc/run.py
```